

# Machine Learning

FRANCESCO RUSSO<sup>1</sup>

October 6, 2026

<sup>1</sup><https://github.com/francescorus>



This document consists of personal, independent, and unofficial lecture notes and summaries covering topics from the **Machine Learning** course for the **Computer Engineering** degree program. This material is created solely to support individual study and is shared free of charge for peer collaboration. Sale or commercial distribution is strictly prohibited.



# Introduction

**Note:** The core course notes officially begin in Chapter 1 (the preceding sections serve as introductory background—feel free to skip them if you wish).

Machine Learning provides frameworks and tools to solve decision-making tasks under limited information provided by measured data. The fundamental components of learning are:

1. **Input**  $x \in \mathcal{X}$ : a vector of  $p$  features from which we make predictions.
2. **Output**  $y \in \mathcal{Y}$ : the target value we want to predict.

Our goal is to learn a target function:

$$f: \mathcal{X} \rightarrow \mathcal{Y}$$

The data consists of a set of examples:

$$S = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

which forms the **training set** (typically assumed to be an i.i.d. sample drawn from a distribution  $\mathcal{D}$ ).

After training on this dataset, our algorithm outputs a **hypothesis function**  $h: \mathcal{X} \rightarrow \mathcal{Y}$  chosen from a hypothesis class  $\mathcal{H}$  ( $h \in \mathcal{H}$ ) that approximates the true function  $f$  as closely as possible.

Machine Learning problems can be categorized into several paradigms:

- **Classification:** Supervised learning where  $\mathcal{Y}$  is discrete.
- **Regression:** Supervised learning where  $\mathcal{Y}$  is continuous.
- **Clustering:** Unsupervised learning where  $\mathcal{Y}$  is discrete.
- **Dimensionality Reduction:** Unsupervised learning where  $\mathcal{Y}$  is continuous.

Before diving deeper into Machine Learning topics, we must review the three core pillars that underpin the field: Probability and Statistics, Linear Algebra and Optimization, and Computation (algorithms and data structures).

## Statistics

### Estimating the Mean of a Gaussian Random Variable

Suppose we observe a dataset of independent and identically distributed (i.i.d.) random variables, and we assume  $X_i \sim \mathcal{N}(\theta, \sigma^2)$ . Our goal is to estimate the unknown mean  $\theta$  using the realizations  $x_1, \dots, x_m$ . To do this, we need to:

1. Find a point estimator  $\hat{\theta} = g(X_1, \dots, X_m)$ .
2. Determine how close this point estimator is to the true mean  $\theta$ .
3. Decide whether it is plausible that  $\theta$  equals a specific value (for example, whether the data supports the claim that  $\theta = 0$ ).

**Probability vs. Likelihood.** **Probability** answers the question: "If the parameter  $\theta$  is known, how likely are we to observe the data  $x$ ?"

**Likelihood** inverts this perspective: "Given that we have observed data  $x = [x_1, \dots, x_m]$  but do not know the true parameter  $\theta$ , which parameter value makes our observed data most plausible?"

The **likelihood function**  $p_X(x; \theta)$  is the joint probability density of the sample vector  $x$  parameterized by the unknown parameter  $\theta$ . For a fixed observation  $x$ , it evaluates how the plausibility of the observed data varies as a function of  $\theta$ .

Because products of probability densities turn into convenient sums under monotonic transformations, we define the **log-likelihood function**:

$$L(\theta) := \log p_X(x; \theta)$$

**Example: Gaussian Mean Estimation.** For  $X_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(\theta, \sigma^2)$ , the density for a single sample is:

$$p_{X_i}(x_i; \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \theta)^2}{2\sigma^2}\right)$$

The log-likelihood of the full sample is therefore:

$$L(\theta) := \log p_X(x; \theta) = \log \left( \prod_{i=1}^m p_{X_i}(x_i; \theta) \right) = -\frac{m}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^m (x_i - \theta)^2$$

(Verifying these algebraic steps is left as a simple exercise).

The **Maximum Likelihood Estimator (MLE)** of  $\theta$  is defined as:

$$\hat{\theta} := \arg \max_{\theta} L(\theta)$$

In words,  $\hat{\theta}$  is the parameter value that maximizes the log-likelihood function  $L(\theta)$ . Searching for this maximum is well motivated because, as the sample size  $m$  grows large, the maximizer converges to the true population mean.

Solving the first-order condition  $\frac{d}{d\theta} L(\theta) = 0$  yields:

$$\hat{\theta} = \frac{1}{m} \sum_{i=1}^m x_i$$

which is simply the sample mean. Note that if we observe only a single measurement  $x_1$ , the value of  $\hat{\theta}$  that maximizes the likelihood is simply  $x_1$  itself.

**Confidence Intervals.** A point estimate alone is not sufficient because the estimator itself is a random variable. If we repeat the experiment tomorrow with a fresh batch of samples, we will obtain a slightly different value for  $\hat{\theta}$ . A single number does not tell us how close our estimate is to the true  $\theta$ .

Instead of a single point, we construct a data-dependent set or interval  $I(X)$  centered around  $\hat{\theta}$  that is likely to contain the true parameter  $\theta$ .

*Important conceptual distinction:* The interval  $I(X)$  is a **random set** (since it depends on the random sample  $X$ ), whereas the parameter  $\theta$  is an **unknown, fixed constant**.

We say that  $I(X)$  is a **confidence interval** for  $\theta$  at confidence level  $1 - \alpha$  if:

$$\Pr[\theta \in I(X)] \geq 1 - \alpha$$

In frequentist terms, this means that across repeated independent experiments drawing samples  $x \sim p_X(x; \theta)$ , approximately  $(1 - \alpha) \times 100\%$  of the constructed intervals will contain the true, unknown parameter  $\theta$ .

Conceptually, the interval can be expressed as:

$$I(X) = \text{Center} \pm \text{Margin of Error}$$

Ideally, we want  $I(X)$  to be narrow (providing high precision) while having a small  $\alpha$  (providing high confidence).

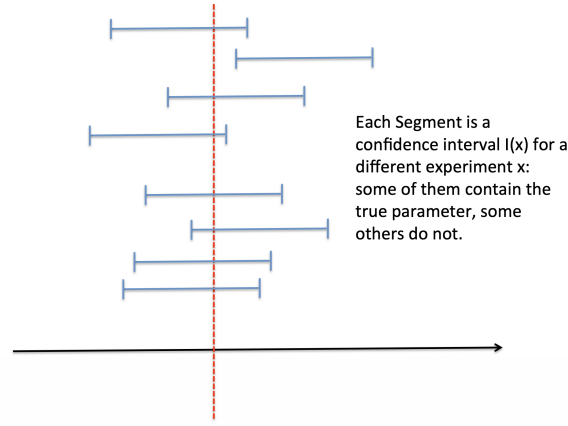


Figure 1: Example of confidence intervals across repeated samples.

Since  $\hat{\theta} = \frac{1}{m} \sum_{i=1}^m X_i \sim \mathcal{N}\left(\theta, \frac{\sigma^2}{m}\right)$ , one can readily show that for a given confidence level  $1 - \alpha$ , the narrowest confidence interval for  $\theta$  is symmetric and centered at the sample mean:

$$I(X) = \left[ \hat{\theta} - z_{\alpha/2} \frac{\sigma}{\sqrt{m}}, \hat{\theta} + z_{\alpha/2} \frac{\sigma}{\sqrt{m}} \right]$$

**Hypothesis Testing.** Often, rather than just estimating  $\theta$ , we want to test a hypothesis about its value (the **null hypothesis**  $H_0$ ). For instance, we may wish to test whether  $\theta = \theta_0$  for some fixed, known value  $\theta_0$ .

Before observing the data, we partition the sample space into two regions:

- **Acceptance Region:** Outcomes that are plausible and expected under  $H_0$ .
- **Rejection Region  $\mathcal{R}(H_0)$ :** Extreme or unlikely outcomes under  $H_0$  that lead us to reject  $H_0$ .

Given an observed sample  $x \sim p_X(x; \theta)$ , our decision rule is:

$$x \in \mathcal{R}(H_0) \implies \text{Reject } H_0$$

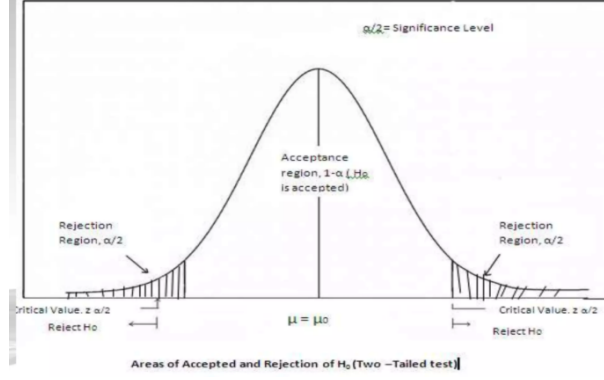


Figure 2: If the true distribution has mean  $\mu_0$ , sample realizations falling within the rejection tails are considered too far from the expected value to be consistent with  $H_0$ .

This decision rule can result in two types of errors:

1. **Type I Error:** Rejecting  $H_0$  when  $H_0$  is true.
2. **Type II Error:** Failing to reject  $H_0$  when  $H_0$  is false.

Their respective probabilities are denoted by:

$$\Pr[x \in \mathcal{R}(H_0) \mid H_0 \text{ is true}] = \alpha \quad (\text{Significance Level})$$

$$\Pr[x \notin \mathcal{R}(H_0) \mid H_0 \text{ is false}] = \beta$$

Evaluating the Type I error probability is typically more straightforward because the null distribution is fully specified. Note that fixing the significance level  $\alpha$  does not uniquely determine the rejection region; it only specifies the total probability mass (e.g., 5%) allocated to the rejection region under the null hypothesis. That probability mass could be placed entirely in the upper tail, entirely in the lower tail, or split equally between both tails (2.5% each).

**Gaussian Test Example.** Returning to our Gaussian mean example (assuming for simplicity that  $\sigma^2 = 1$ ), suppose we test the null hypothesis:

$$H_0: \theta = \theta_0 = 0$$

The Neyman-Pearson lemma ensures that the most powerful symmetric two-sided rejection region for a fixed Type I error rate  $\alpha$  has the form:

$$\mathcal{R}(H_0) = \left\{ (x_1, \dots, x_m) : |\hat{\theta} - \theta_0| = \left| \frac{1}{m} \sum_{i=1}^m (x_i - \theta_0) \right| > \frac{z_{\alpha/2}}{\sqrt{m}} \right\}$$

Setting  $\theta_0 = 0$ , this simplifies to:

$$\mathcal{R}(H_0) = \left\{ (x_1, \dots, x_m) : |\hat{\theta}| = \left| \frac{1}{m} \sum_{i=1}^m x_i \right| > \frac{z_{\alpha/2}}{\sqrt{m}} \right\}$$

Since under  $H_0$  we have  $\hat{\theta} \sim \mathcal{N}\left(0, \frac{1}{m}\right)$ , the probability of committing a Type I error directly matches our target significance level  $\alpha$ :

$$\Pr \left[ |\hat{\theta}| > \frac{z_{\alpha/2}}{\sqrt{m}} \mid H_0 \right] = \alpha$$

**The  $p$ -value.** Instead of comparing the test statistic directly to a predetermined critical threshold, we can summarize the test using the  $p$ -value.

Given the observed sample mean  $\bar{x} = \frac{1}{m} \sum_{i=1}^m x_i$ , the  $p$ -value is the probability of observing a test statistic at least as extreme as the one computed from the data, assuming  $H_0$  is true:

$$p := \Pr \left[ |Z| > \left| \frac{1}{m} \sum_{i=1}^m x_i \right| \middle| H_0 \right], \quad \text{where } Z \sim \mathcal{N} \left( 0, \frac{1}{m} \right)$$

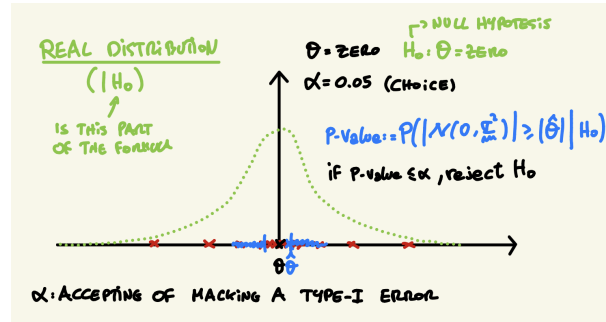


Figure 3: Visual representation of the two-tailed  $p$ -value area under the null distribution.

**Duality Between Confidence Intervals and Hypothesis Tests.** There is a direct mathematical duality connecting these two concepts: rejecting the null hypothesis  $H_0: \theta = \theta_0$  at significance level  $\alpha$  is completely equivalent to checking whether the hypothesized value  $\theta_0$  falls outside the corresponding  $(1 - \alpha)$  confidence interval  $I(X)$ .

### Linear Regression with Known Variance $\sigma^2$

*To be continued...*



# Chapter 1

In supervised learning, our goal is to infer a rule from observed data. We start by formally defining the spaces:

- **Domain Set ( $\mathcal{X}$ ):** The space of all possible inputs (features).
- **Outout Set (label set for classification problem) ( $\mathcal{Y}$ ):** The space of all possible outputs (targets). Depending on the problem,  $\mathcal{Y}$  can be:
  - *Discrete*: in classification problems (e.g., recognizing handwritten digits from an image).
  - *Continuous*: in regression problems.

The learner does not see the entire world, but only a finite set of labeled examples called the **Training Set** (or sample set):

$$S := \{(x_i, y_i)\}_{i=1}^m = \{z_i\}_{i=1}^m$$

where  $m$  denotes the sample size.

## How does the environment generate data?

To analyze learning mathematically, we must make assumptions about how the training set  $S$  is produced:

1. **Simple Assumption (Deterministic Labels):** Each instance  $x_i$  is sampled independently and identically distributed (i.i.d.) from a probability distribution  $\mathcal{D}_x$ , and there exists an unknown deterministic ground-truth function  $f(x)$  that maps each  $x_i$  to its exact label:

$$x_i \sim \mathcal{D}_x \quad \text{and} \quad \exists f(x) \text{ such that } y_i = f(x_i)$$

2. **Realistic Assumption (Probabilistic Labels):** In the real world, identical inputs may have different labels due to noise or missing information. Thus, the output  $y$  is also governed by a distribution conditioned on  $x$ :

$$y \mid x \sim \mathcal{D}_{y|x}$$

**Key Insight:** The *Realistic* setting is the general framework. The *Simple* setting is just a special case where  $\mathcal{D}_{y|x}$  is completely deterministic—meaning all the probability mass is concentrated at the single value  $f(x)$  (conceptually like a Dirac delta  $\delta$  function).

## 2. The Learning Algorithm and Loss Functions

Under the deterministic assumption, our concrete goal is: **learn  $f(x)$  from empirical measurements to predict  $y$  for unseen inputs  $x$ .**

A **Learning Algorithm**  $A$  takes the training set  $S$  as input and outputs a hypothesis function  $h$  from a hypothesis class  $\mathcal{H}$ :

$$A : S \rightarrow h \in \mathcal{H}$$

Here,  $h$  acts as our best candidate approximation of the true function  $f$ .

### Quantifying Error: The Loss Function

To measure how well a chosen hypothesis  $h$  predicts  $y$ , we define a **Loss Function**  $\ell(h, z)$ .

*Important distinction:* The loss function evaluates the discrepancy on **a single sample  $\mathbf{z}=(\mathbf{x},\mathbf{y})!$**   $(x, y) \sim D_{x,y} = D_{y|x} \cdot D_x$ , in the simple assumption  $D_{y|x} = f(x)$

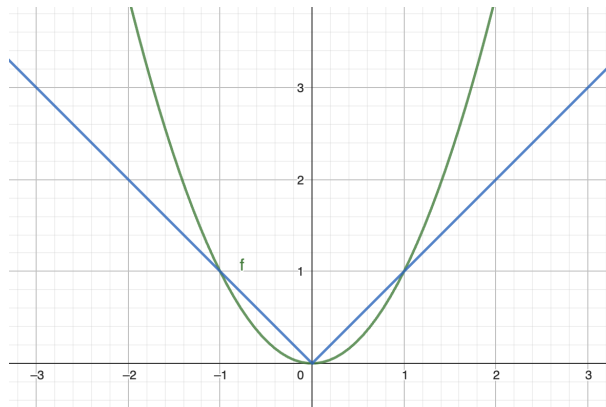
#### Example 1: Classification

The most standard choice in classification is the **0–1 loss function**:

$$y \in \{0, 1\}, \quad \ell_{0-1}(h, z) = \begin{cases} 0 & \text{if } h(x) = y \quad (\text{correct classification}) \\ 1 & \text{if } h(x) \neq y \quad (\text{misclassification}) \end{cases} \quad (1)$$

#### Example 2: Quadratic Loss vs. Absolute Loss

The mathematical choice of the loss function heavily affects how the algorithm treats mistakes:



Consider comparing:

$$\text{Squared Error: } \ell(h, z) = (y - h(x))^2 \quad (y \in \mathbb{R})$$

vs.

$$\text{Absolute Error (Modulus): } \ell(h, z) = |y - h(x)| \quad (y \in \mathbb{R})$$

#### Takeaway for students:

- The **quadratic loss** penalizes larger errors much more heavily (growing quadratically), making it sensitive to outliers.
- The **modulus loss** increases linearly with the error magnitude, maintaining constant sensitivity regardless of scale.

### 3. From Single Samples to Global Error: True Risk vs. Empirical Risk

Evaluating the error on a single instance  $(x, y)$  is not enough: we want to know how well  $h$  performs on average across all possible instances in the environment. This average is the **True Risk** (or Generalization Error):

$$L_{\mathcal{D}}(h) := \mathbb{E}_{z=(x,y) \sim \mathcal{D}}[\ell(h, z)]$$

$L_{\mathcal{D}}(h)$  represents the true, expected error of  $h$  over the entire distribution  $\mathcal{D}$ . Therefore, the ideal learning algorithm would pick the hypothesis that minimizes this quantity:

$$A(S) = \arg \min_{h \in \mathcal{H}} L_{\mathcal{D}}(h)$$

#### The Big Problem

To compute  $L_{\mathcal{D}}(h) = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\ell(h, (x, y))]$ , we would need complete knowledge of the underlying distribution  $\mathcal{D}$ . But in real-world machine learning,  $\mathcal{D}$  is **unknown**!

#### The Solution: Empirical Risk

Since we cannot compute the theoretical expectation directly, we compute the empirical sample mean over our available training data  $S$ .

By the **Law of Large Numbers (LLN)**, as the sample size grows to infinity ( $m \rightarrow +\infty$ ), the sample mean  $\hat{\theta}$  converges to the true population mean  $\theta$ :

$$\hat{\theta} \approx \theta$$

This justifies calculating the **Empirical Risk**  $L_S(h)$  directly from our dataset:

$$L_S(h) := \frac{1}{m} \sum_{i=1}^m \ell(h, (x_i, y_i))$$

—

### 4. The Ultimate Goal: PAC Learning

Our ultimate goal is **not** just to minimize  $L_S(h)$  on the training data (doing so blindly can lead to *overfitting*, which we will analyze later), but to guarantee that the **True Risk**  $L_{\mathcal{D}}(h)$  is **as small as possible**.

Because the training set  $S$  is drawn randomly, there is always a tiny probability of drawing an unrepresentative sample. Therefore, we look for guarantees of the form:

$$\mathbb{P}[\text{True Risk } L_{\mathcal{D}}(A(S)) \text{ is small}] \text{ is large}$$

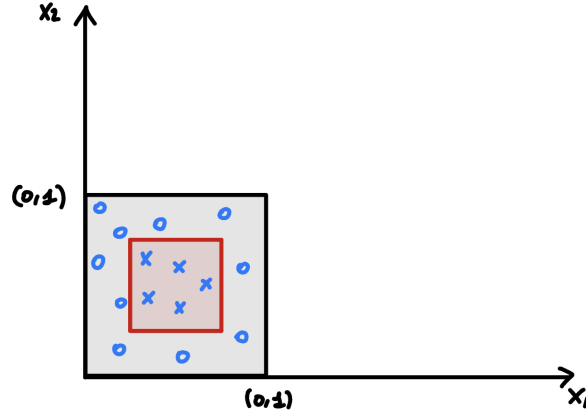
In mathematical terms, we want:

$$\mathbb{P}[L_{\mathcal{D}}(A(S)) \leq \epsilon] \geq 1 - \delta$$

We want that the **probability that True risk is small, is large!** This core framework is known as **PAC Learning** (*Probably Approximately Correct Learning*):

- **Approximately Correct:** The error  $L_{\mathcal{D}}(h)$  is bounded by a small threshold  $\epsilon$ .
- **Probably:** This condition holds with high confidence  $1 - \delta$ .

We don't want  $L_S(\hat{h}_S) = 0$  and  $L_{\mathcal{D}}(\hat{h}_S)$  large. Let's give an example where PAC learning fails without inductive bias:



The true function in this case that maps every input  $x \in \mathcal{X}$  to  $y = f(x)$  is:

$$y = \begin{cases} 1 & \text{if } x \in A \quad (\text{classified as "x"}) \\ 0 & \text{if } x \notin A \quad (\text{classified as "o"}) \end{cases} \quad (2)$$

$x \sim \mathcal{U}([0,1] \times [0,1])$ . We would like to take a function that performs well in our dataset and approximates perfectly the function  $f$ , for example:

$$h(x) = \begin{cases} y_i & \text{if } x = x_i \in S \\ 0 & \text{if } x \notin S \end{cases} \quad (3)$$

This equation brings to  $L_S(h) = 0$ , but it matches  $f(x)$  ONLY in the points that we KNOW (in our dataset). We choose from an unrestricted hypothesis space  $\mathcal{H} = \{\text{all possible } h : \mathcal{X} \rightarrow \{0,1\}\}$  one of the  $h(x)$  that minimizes  $L_S(h)$ , as discussed earlier ( $\hat{h}_S \in \arg \min_h L_S(h)$ ). Since we know the true distribution, if  $L_S(\hat{h}_S) = 0$ , what about  $L_{\mathcal{D}}(\hat{h}_S)$ ? Since the training sample  $S$  is finite and the domain is continuous,  $\mathbb{P}_{x \sim \mathcal{D}}(x \in S) = 0$ . For any unseen point, the classifier outputs 0. If  $\mathbb{P}(x \in A) = \frac{1}{2}$ , we find that the true risk is  $\mathbb{P}(x \notin A) \cdot 0 + \mathbb{P}(x \in A) \cdot 1 = \frac{1}{2}$ .

Really bad! We have  $\hat{h}_S$  that overfits the dataset ( $L_S(\hat{h}_S) = 0$ , so no empirical error), but we are classifying correctly only 50% of the time, terrible! If we do not restrict our choice of hypotheses, an algorithm can simply memorize the exact training points. To ensure real learning, we must restrict our search to a specific Hypothesis Class  $\mathcal{H}$  (e.g., linear classifiers, decision trees).

We need to perform well with the data that we have, and try to ensure  $L_{\mathcal{D}}(\hat{h}_S)$  is small. We just saw a case where the empirical risk is zero while the true risk is 0.5. We want to guarantee the true risk of  $\hat{h}_S$  close to the empirical risk of  $\hat{h}_S$  (computable).  $L_{\mathcal{D}}(\hat{h}_S) - L_S(\hat{h}_S) = \text{generalization gap} \rightarrow \text{small}$ . The assumption is that  $L_S(\hat{h}_S) = 0$ . Let's give a formalization:

$$\begin{cases} \text{Want } L_{\mathcal{D}}(\hat{h}_S) \leq \epsilon \\ \mathbb{P}_{S \sim \mathcal{D}^m} [L_{\mathcal{D}}(\hat{h}_S) \leq \epsilon] \geq 1 - \delta \end{cases} \quad (4)$$

Since our training sample  $S$  is drawn randomly ( $S \sim \mathcal{D}^m$ ), we can never guarantee 100% perfection. Instead, we demand two guarantees: **Approximately Correct**: The true error  $L_{\mathcal{D}}(\hat{h}_S)$  is small ( $\leq \epsilon$ , where  $\epsilon$  is the error tolerance, e.g., 0.05), and **Probably**: This happens with high

probability ( $\geq 1 - \delta$ , where  $\delta$  is the failure risk, e.g., 0.01).

### P.A.C. Learnability

Assumption:  $\exists f$  s.t.  $y = f(x)$ .

Assumption:  $\exists h \in \mathcal{H}$  s.t.  $L_{\mathcal{D}}(h) = 0$  (The Realizability Assumption: because a perfect function  $h^*$  exists in  $\mathcal{H}$ , any ERM algorithm that minimizes empirical error will always find a hypothesis  $\hat{h}_S$  with zero training error,  $L_S(\hat{h}_S) = 0$ ).

**Def:**  $\mathcal{H}$  is P.A.C. Learnable if  $\exists m_{\mathcal{H}}(\epsilon, \delta) : (0, 1)^2 \rightarrow \mathbb{N}$ , and  $\exists \mathcal{A}(S)$  s.t.  $\forall \epsilon, \delta \in (0, 1), \forall \mathcal{D}, \forall f \in \mathcal{H}$ , if  $m \geq m_{\mathcal{H}}(\epsilon, \delta)$ , then  $\mathbb{P}_{S \sim \mathcal{D}^m} [L_{\mathcal{D}}(\mathcal{A}(S)) \leq \epsilon] \geq 1 - \delta$ . Therefore hypothesis class  $\mathcal{H}$  is PAC Learnable if, given enough training examples  $m$ , the algorithm produces a hypothesis  $\hat{h}_S$  whose true risk is bounded by  $\epsilon$  with high probability ( $1 - \delta$ ):

$$\mathbb{P}_{S \sim \mathcal{D}^m} (L_{\mathcal{D}}(\hat{h}_S) \leq \epsilon) \geq 1 - \delta \iff \mathbb{P}_{S \sim \mathcal{D}^m} (L_{\mathcal{D}}(\hat{h}_S) > \epsilon) \leq \delta$$

**Theorem:** Assume  $\mathcal{H} = \{h_1, h_2, \dots, h_k\}$  is finite, with  $k = |\mathcal{H}|$ . Under the realizability assumption, if  $m \geq \frac{1}{\epsilon} \ln \left( \frac{|\mathcal{H}|}{\delta} \right) = \frac{1}{\epsilon} \left( \ln |\mathcal{H}| + \ln \frac{1}{\delta} \right)$ , any consistent ERM algorithm  $\mathcal{A}(S)$  satisfies  $\mathbb{P}_{S \sim \mathcal{D}^m} [L_{\mathcal{D}}(\mathcal{A}(S)) > \epsilon] \leq \delta$ .

So before the experiment, and before collecting data, if we are under the P.A.C. learnability hypothesis, we choose  $\epsilon$  and  $\delta$  and then find how many samples  $m$  we need in order to achieve that probability guarantee.

**Proof:** Remark: we are under the realizability assumption, so there exists  $h^* \in \mathcal{H}$  with  $L_{\mathcal{D}}(h^*) = 0 \implies L_S(\hat{h}_S) = 0$ .

First, we define the set of “bad” hypotheses by collecting all candidate functions in  $\mathcal{H}$  whose true population error exceeds our tolerance threshold  $\epsilon$ :

$$\mathcal{H}_{\text{bad}} \triangleq \{h \in \mathcal{H} : L_{\mathcal{D}}(h) > \epsilon\}$$

Our learning algorithm fails whenever it outputs a model  $\hat{h}_S \in \mathcal{H}_{\text{bad}}$ . Under the realizability assumption, the ERM algorithm always finds a model with  $L_S(\hat{h}_S) = 0$ . Thus, a failure ( $\hat{h}_S \in \mathcal{H}_{\text{bad}}$ ) can *only* occur if a bad hypothesis accidentally gets zero errors on our training sample  $S$ .

We define the set of *misleading samples*  $M$  as those datasets that trick the algorithm:

$$M \triangleq \{S \in \mathcal{X}^m : \exists h \in \mathcal{H}_{\text{bad}} \text{ such that } L_S(h) = 0\}$$

Because every failure dataset  $S$  produces an ERM model that is in  $\mathcal{H}_{\text{bad}}$  and satisfies  $L_S(\hat{h}_S) = 0$ , every failure sample necessarily belongs to  $M$ :

$$\{S : L_{\mathcal{D}}(\hat{h}_S) > \epsilon\} = \{S : \hat{h}_S \in \mathcal{H}_{\text{bad}}\} \subseteq M = \bigcup_{h \in \mathcal{H}_{\text{bad}}} \{S : L_S(h) = 0\}$$

By applying probability monotonicity ( $A \subseteq B \implies \mathbb{P}(A) \leq \mathbb{P}(B)$ ) and Boole’s Inequality (the Union Bound) to these sets, we can bound the failure probability:

$$\mathbb{P}_{S \sim \mathcal{D}^m} (L_{\mathcal{D}}(\hat{h}_S) > \epsilon) \leq \mathbb{P}_{S \sim \mathcal{D}^m}(M) \leq \sum_{h \in \mathcal{H}_{\text{bad}}} \mathbb{P}_{S \sim \mathcal{D}^m} (L_S(h) = 0)$$

To bound the probability for a single bad hypothesis, we pick a fixed  $h \in \mathcal{H}_{\text{bad}}$ . Because  $L_{\mathcal{D}}(h) > \epsilon$ , its probability of correctly classifying one random training instance is strictly less than  $1 - \epsilon$ . Since all  $m$  training instances are drawn independently (i.i.d.), its probability of correctly classifying *all*  $m$  instances is:

$$\mathbb{P}_{S \sim \mathcal{D}^m} (L_S(h) = 0) < (1 - \epsilon)^m$$

Using the algebraic inequality  $1 - x \leq e^{-x}$ , we can upper-bound this term:

$$\mathbb{P}_{S \sim \mathcal{D}^m}(L_S(h) = 0) < e^{-m\epsilon}$$

Substituting  $e^{-m\epsilon}$  back into our Union Bound sum gives:

$$\mathbb{P}_{S \sim \mathcal{D}^m}(\hat{h}_S \in \mathcal{H}_{\text{bad}}) < \sum_{h \in \mathcal{H}_{\text{bad}}} e^{-m\epsilon} = |\mathcal{H}_{\text{bad}}| \cdot e^{-m\epsilon}$$

Since the set of bad hypotheses is a subset of the full hypothesis space ( $\mathcal{H}_{\text{bad}} \subseteq \mathcal{H}$ ), we can bound its cardinality by  $|\mathcal{H}_{\text{bad}}| \leq |\mathcal{H}|$ , which leaves us with:

$$\mathbb{P}_{S \sim \mathcal{D}^m}(L_{\mathcal{D}}(\hat{h}_S) > \epsilon) \leq |\mathcal{H}| \cdot e^{-m\epsilon}$$

Finally, to guarantee that the overall failure probability is bounded by  $\delta$ , we set:

$$|\mathcal{H}| \cdot e^{-m\epsilon} \leq \delta$$

Dividing both sides by  $|\mathcal{H}|$ :

$$e^{-m\epsilon} \leq \frac{\delta}{|\mathcal{H}|}$$

Taking the natural logarithm ( $\ln$ ) of both sides yields:

$$-m\epsilon \leq \ln\left(\frac{\delta}{|\mathcal{H}|}\right) = \ln \delta - \ln |\mathcal{H}|$$

Multiplying by  $-1$  (which flips the inequality from  $\leq$  to  $\geq$ ) and dividing by  $\epsilon$  gives the minimal sample size  $m$ :

$$\mathbf{m} \geq \frac{1}{\epsilon} \left( \ln |\mathcal{H}| + \ln \left( \frac{1}{\delta} \right) \right) \quad \blacksquare$$